

Blind Signatures from Oblivious Transfer : A Detailed Analysis

Julian Mouthon

LIP6 — Sorbonne Université

Supervisé par Ky Nguyen

22 juillet 2026

Résumé

Les signatures aveugles (Blind Signatures) introduites par Chaum en 1982, permettent à un utilisateur d'obtenir une signature sur un message déterminé sans que le signataire n'apprenne son contenu. Ces mécanismes peuvent être utilisés en pratique dans le cadre des votes électroniques, des transactions bancaires ou des protocoles d'identification.

Ce rapport présente un nouveau schéma de signatures aveugles utilisant le transfert inconscient (Oblivious Transfer, OT) et les signatures de Waters.

Table des matières

1	Préliminaires	3
1.1	Notations	3
1.2	Groupes bilinéaires	3
1.3	Fonction de hachage de Waters	3
1.4	Schéma de signature de Waters	3
1.5	Signatures aveugles	4
2	Protocole basé sur OTs	4
2.1	Paramètres publics	4
2.2	Schéma de signature	5
2.3	Rôle de l'Oblivious Transfer	5
2.4	Rôle du chiffrement d'ElGamal	6
2.5	Correctness	7
2.6	Aveuglement sous clés malveillantes	8
2.6.1	Définition	8
2.6.2	Preuve : Blindness du schéma de signature	8
2.6.3	Preuve : randomisation de Waters ($G_1 \rightarrow G_2$)	10
2.6.4	Preuve : Réduction DDH ($G_1 \rightarrow G_2$)	10
3	Protocole basé sur Dual-Mode OTs	10
3.1	Paramètres publics	10
3.2	Schéma de signature	11
3.3	Correctness	11
3.4	Branches DH et messy	12

1 Préliminaires

Cette section introduit les notations et les objets cryptographiques nécessaires à la compréhension de la construction étudiée.

1.1 Notations

Les notations suivantes seront utilisées tout au long du rapport :

- λ désigne le paramètre de sécurité.
- Un algorithme probabiliste en temps polynomial (PPT) est un algorithme dont le temps d'exécution est polynomial en λ .
- Pour un ensemble fini S , la notation $x \xleftarrow{\$} S$ désigne un tirage uniforme de x dans S .
- Pour $\ell \in \mathbb{N}$, on note $[\ell] = \{1, \dots, \ell\}$.
- Une fonction $f(\lambda)$ est dite négligeable, notée $f(\lambda) = \text{negl}(\lambda)$, si elle décroît plus rapidement que l'inverse de tout polynôme en λ .
- $\perp$ désigne l'élément d'échec retourné par un algorithme.

1.2 Groupes bilinéaires

La construction repose sur un système de groupes bilinéaires. Soient $\mathbb{G}_1$, $\mathbb{G}_2$ et $\mathbb{G}_T$ trois groupes cycliques d'ordre premier p . Un couplage bilinéaire est une application :

$$e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$$

satisfaisant les propriétés suivantes :

- **Bilinéarité** : pour tout $g_1 \in \mathbb{G}_1$, $g_2 \in \mathbb{G}_2$ et $a, b \in \mathbb{Z}_p$,

$$e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}.$$

- **Non-dégénérescence** : si g_1 et g_2 sont des générateurs, alors

$$e(g_1, g_2) \neq 1_{\mathbb{G}_T}.$$

- **Efficacité** : le calcul du couplage est réalisable en temps polynomial.

1.3 Fonction de hachage de Waters

La construction utilise la fonction de hachage de Waters permettant d'encoder un message binaire dans un élément du groupe $\mathbb{G}_1$. Soit un message $M = (M_1, \dots, M_\ell) \in \{0, 1\}^\ell$. Les paramètres publics de la fonction sont constitués de $u_0, u_1, \dots, u_\ell \in \mathbb{G}_1$. La valeur associée au message est définie par :

$$F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}.$$

Ainsi, chaque bit M_i influence la valeur de hachage par l'intermédiaire du facteur $u_i^{M_i}$.

1.4 Schéma de signature de Waters

Le schéma de signature de Waters utilise les paramètres publics précédents ainsi qu'une clé secrète $a \in \mathbb{Z}_p$. La clé publique est $\text{bvk} = g_2^a$. Pour signer un message M , le signataire choisit un aléa $t \xleftarrow{\$} \mathbb{Z}_p$ et

calcule :

$$\sigma_1 = h_s^a F(M)^t, \quad \sigma_2 = g_2^t.$$

La signature est alors

$$\sigma = (F(M), \sigma_1, \sigma_2).$$

La vérification consiste à vérifier l'équation de couplage

$$e(h_s, \mathbf{bvk}) \cdot e(F(M), \sigma_2) \stackrel{?}{=} e(\sigma_1, g_2).$$

1.5 Signatures aveugles

Une signature aveugle permet à un utilisateur d'obtenir une signature sur un message sans révéler ce message au signataire. Un schéma de signature aveugle est constitué des algorithmes suivants :

- $\text{BSGen}(1^\lambda)$: génère une clé publique $\mathbf{bvk}$ et une clé secrète $\mathbf{bsk}$;
- $\text{BSUser}(\mathbf{bvk}, M)$: exécuté par l'utilisateur, produit un premier message ρ_1 ainsi qu'un état interne st ;
- $\text{BSSigner}(\mathbf{bsk}, \rho_1)$: exécuté par le signataire, produit une réponse ρ_2 ;
- $\text{BSDerive}(st, \rho_2)$: permet à l'utilisateur d'obtenir une signature σ ;
- $\text{BSVerify}(\mathbf{bvk}, M, \sigma)$: vérifie la validité de la signature.

La propriété de correction exige qu'une exécution honnête produise une signature valide :

$$\Pr[\text{BSVerify}(\mathbf{bvk}, M, \sigma) = 1] \geq 1 - \text{negl}(\lambda).$$

La propriété d'aveuglement garantit que le signataire ne peut pas déterminer quel message a été signé parmi plusieurs exécutions possibles.

2 Protocole basé sur OTs

2.1 Paramètres publics

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$.
- un vecteur $\mathbf{ek} = (h_i)_{i=1}^\ell$ avec $h_i = g_1^{x_i}$ obtenu à partir de scalaires aléatoires $x_i \xleftarrow{\$} \mathbb{Z}_p$.
- la clé publique du schéma de signature $\mathbf{bvk} = g_2^a$, associée à la clé secrète $\mathbf{bsk} = h_s^a$, avec $h_s = \prod_{i=1}^\ell h_i$.

2.2 Schéma de signature

Signer	User
	$\forall i \in [\ell] : y_i \xleftarrow{\$} \mathbb{Z}_p$ $\text{ek}_{i,M_i} = g_1^{y_i}$ $\text{ek}_{i,1-M_i} = g_1^{x_i - y_i}$ $\xleftarrow{(\text{ek}_{i,0}, \text{ek}_{i,1})_{i \in [\ell]}}$
$\forall i \in [\ell] : \text{ek}_{i,0} \cdot \text{ek}_{i,1} = g_1^{x_i}$ $\forall i \in [\ell] : \text{choose } \alpha_i \text{ s.t. } \prod_i \alpha_i = h_s^a$ $r, t \xleftarrow{\$} \mathbb{Z}_p$ $\forall i \in [\ell] : \begin{aligned} \text{ct}_{i,0} &\leftarrow \text{ek}_{i,0}^r \cdot \alpha_i, \\ \text{ct}_{i,1} &\leftarrow \text{ek}_{i,1}^r \cdot \alpha_i(u_i)^t \end{aligned}$ $\text{ct}_0 = g_1^r, \quad \sigma_2 = g_2^t$ $\xrightarrow{(\text{ct}_{i,0}, \text{ct}_{i,1})_{i \in [\ell]}, \text{ct}_0, \sigma_2}$	$\sigma_{1,i} = \frac{\text{ct}_{i,M_i}}{\text{ct}_0^{y_i}}$ $\sigma_1 = \prod_{i \in [\ell]} \sigma_{1,i}$ $t' \xleftarrow{\$} \mathbb{Z}_p$ $\hat{\sigma}_1 = \sigma_1 \cdot F(M)^{t'}$ $\hat{\sigma}_2 = \sigma_2 \cdot g_2^{t'}$ return $\sigma = (F(M), \hat{\sigma}_1, \hat{\sigma}_2)$

2.3 Rôle de l'Oblivious Transfer

L'utilisation du transfert inconscient permet d'intégrer le message dans la signature sans que le signataire n'apprenne les bits qui le composent. Pour chaque bit M_i du message M , deux branches sont construites :

$$0 \longrightarrow \alpha_i, \quad 1 \longrightarrow \alpha_i u_i.$$

où α_i est un facteur aléatoire choisi par le signataire et u_i est le i -ème paramètre de la fonction de hachage de Waters. Ces deux branches sont ensuite masquées dans une structure compatible avec le chiffrement d'ElGamal à l'aide des clés d'évaluation

$$\text{ek}_{i,M_i} = g_1^{y_i}, \quad \text{ek}_{i,1-M_i} = g_1^{x_i - y_i},$$

où y_i est choisi uniformément dans $\mathbb{Z}_p$. Cette construction garantit que l'utilisateur récupère uniquement la branche correspondant au bit M_i , sans révéler celui-ci au signataire. Après déchiffrement, l'utilisateur obtient pour chaque position i , la valeur

$$\alpha_i(u_i^{M_i})^t.$$

En multipliant toutes les composantes, il calcule

$$\begin{aligned}\sigma_1 &= \prod_{i=1}^{\ell} \alpha_i (u_i^{M_i})^t \\ &= \left(\prod_{i=1}^{\ell} \alpha_i \right) \left(\prod_{i=1}^{\ell} u_i^{M_i} \right)^t.\end{aligned}$$

Le premier facteur correspond au masquage par aléas, et le second à la partie dépendante du message dans la fonction de hachage de Waters. Ainsi, les bits M_i sont encodés par le choix entre les deux branches α_i et $\alpha_i u_i$. Cette sélection introduit les facteurs $u_i^{M_i}$ dans la signature sans jamais révéler au signataire les valeurs des bits du message.

Le rôle du bit M_i apparaît dans l'affectation des deux clés d'évaluation. Si $M_i = 0$, l'utilisateur construit $(\text{ek}_{i,0} = g_1^{y_i}, \text{ek}_{i,1} = g_1^{x_i - y_i})$, si $M_i = 1$, il inverse les deux valeurs $(\text{ek}_{i,1} = g_1^{y_i}, \text{ek}_{i,0} = g_1^{x_i - y_i})$. Le signataire reçoit toujours le même couple $(\text{ek}_{i,0}, \text{ek}_{i,1})$ sans pouvoir distinguer lequel des deux éléments contient l'exposant y_i . Donc aucune information sur le bit M_i n'est révélée. Seule l'affectation des deux composantes dépend du message. Il calcule ensuite

$$\text{ct}_{i,0} = \text{ek}_{i,0}^r \alpha_i, \quad \text{ct}_{i,1} = \text{ek}_{i,1}^r \alpha_i (u_i)^t.$$

Lors du déchiffrement, l'utilisateur récupère la composante correspondant à son bit et élimine le terme aléatoire en calculant

$$\sigma_{1,i} = \frac{\text{ct}_{i,M_i}}{\text{ct}_0^{y_i}}.$$

Cas $M_i = 0$:

$$\text{ct}_{i,0} = (g_1^{y_i})^r \alpha_i = g_1^{ry_i} \alpha_i.$$

Donc

$$\sigma_{1,i} = \frac{\text{ct}_{i,0}}{\text{ct}_0^{y_i}} = \frac{g_1^{ry_i} \alpha_i}{(g_1^r)^{y_i}} = \frac{g_1^{ry_i} \alpha_i}{g_1^{ry_i}} = \alpha_i.$$

Cas $M_i = 1$:

$$\text{ct}_{i,1} = (g_1^{y_i})^r \alpha_i (u_i)^t = g_1^{ry_i} \alpha_i (u_i)^t$$

Donc

$$\sigma_{1,i} = \frac{\text{ct}_{i,1}}{\text{ct}_0^{y_i}} = \frac{g_1^{ry_i} \alpha_i (u_i)^t}{(g_1^r)^{y_i}} = \frac{g_1^{ry_i} \alpha_i (u_i)^t}{g_1^{ry_i}} = \alpha_i (u_i)^t$$

Ainsi, l'Oblivious Transfer permet donc de réaliser une sélection privée entre les deux branches associées à chaque bit du message. Le signataire construit les deux possibilités α_i et $\alpha_i u_i$ mais ne peut pas apprendre laquelle sera finalement utilisée par l'utilisateur. Cette propriété permet d'intégrer directement le message M dans la signature de Waters tout en préservant l'aveuglement du protocole.

2.4 Rôle du chiffrement d'ElGamal

Dans cette construction, le chiffrement d'ElGamal est utilisé comme masquage temporaire permettant au signataire de transmettre les deux valeurs possibles associées à chaque bit du message sans révéler laquelle sera sélectionnée par l'utilisateur.

$$\begin{array}{l}
\textbf{Signer} \\
\hline
r, t \xleftarrow{\$} \mathbb{Z}_p \\
\forall i \in [\ell] : \quad \text{ct}_{i,0} \leftarrow \text{ek}_{i,0}^r \cdot \alpha_i, \\
\quad \quad \quad \text{ct}_{i,1} \leftarrow \text{ek}_{i,1}^r \cdot \alpha_i(u_i)^t \\
\text{ct}_0 = g_1^r, \quad \sigma_2 = g_2^t \\
\hline
(\text{ct}_{i,0}, \text{ct}_{i,1})_{i \in [\ell]}, \text{ct}_0, \sigma_2 \\
\hline
\end{array}$$

FIGURE 1 – Masquage des branches par ElGamal

Les deux valeurs $m_{i,0} = \alpha_i$, $m_{i,1} = \alpha_i u_i$ peuvent être vues comme les deux messages chiffrés par ElGamal. Par comparaison avec un chiffrement ElGamal classique $(g^r, h^r \cdot M)$, notre construction utilise une structure similaire :

$$(\text{ct}_0, \text{ct}_{i,M_i}) = (g_1^r, \text{ek}_{i,M_i}^r \cdot \alpha_i(u_i^{M_i})^t).$$

Ici, les deux messages possibles sont

$$\alpha_i \quad \text{si } M_i = 0, \quad \alpha_i u_i \quad \text{si } M_i = 1.$$

L'exposant t permet de randomiser ces deux valeurs avant leur transmission. Ainsi, ElGamal masque temporairement les valeurs α_i et $\alpha_i u_i$ tout en permettant à l'utilisateur de récupérer uniquement la branche correspondant à son bit M_i .

2.5 Correctness

La signature obtenue par l'utilisateur est une signature de Waters de la forme $\sigma = (F(M), \hat{\sigma}_1, \hat{\sigma}_2)$ avec :

$$\hat{\sigma}_1 = \sigma_1 \cdot F(M)^{t'} = h_s^a \cdot F(M)^{t+t'}, \quad \hat{\sigma}_2 = \sigma_2 \cdot g_2^{t'} = g_2^{t+t'}.$$

On remarque que le signataire ne connaît pas la valeur $t + t'$, car le terme t' est choisi aléatoirement par l'utilisateur après la phase de signature. La vérification s'effectue ensuite à l'aide de la clé publique du signataire $\text{bvk} = g_2^a$ en vérifiant l'équation de couplage :

$$e(h_s, \text{bvk}) \cdot e(F(M), \hat{\sigma}_2) \stackrel{?}{=} e(\hat{\sigma}_1, g_2).$$

En remplaçant les valeurs obtenues par l'utilisateur :

$$\begin{aligned}
e(h_s, \text{bvk}) \cdot e(F(M), \hat{\sigma}_2) &= e(h_s, g_2^a) \cdot e(F(M), g_2^{t+t'}) \\
&= e(h_s^a, g_2) \cdot e(F(M)^{t+t'}, g_2) \quad \text{par bilinéarité du couplage} \\
&= e(h_s^a \cdot F(M)^{t+t'}, g_2) \quad \text{par bilinéarité et multiplicativité} \\
&= e(\hat{\sigma}_1, g_2)
\end{aligned}$$

Ainsi, l'équation de vérification est satisfaite et la signature obtenue est une signature de Waters valide.

2.6 Aveuglement sous clés malveillantes

2.6.1 Définition

Definition 2.1 (Blindness). Un schéma de signature aveugle est dit aveugle sous clés malveillantes si pour tout adversaire PPT $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}}^{\text{blind}}(\lambda) = \Pr \left[\begin{array}{l} (\text{bvk}, m_0, m_1) \leftarrow \mathcal{A}(1^\lambda), \ b \xleftarrow{\$} \{0, 1\}, \\ (\rho_{1,i}, st_i) \leftarrow \text{BSUser}(\text{bvk}, m_i) \quad (i \in \{0, 1\}), \\ (\rho_{2,b}, \rho_{2,1-b}) \leftarrow \mathcal{A}(\rho_{1,b}, \rho_{1,1-b}), \\ \sigma_i \leftarrow \text{BSDerive}(st_i, \rho_{2,i}) \quad (i \in \{0, 1\}), \\ \text{if } \exists i, \text{BSVerify}(\text{bvk}, m_i, \sigma_i) = 0 : \\ \quad \text{then } \sigma_0 = \sigma_1 = \perp, \\ b = \mathcal{A}(\sigma_0, \sigma_1) \end{array} \right] - \frac{1}{2} = \text{negl}(\lambda).$$

Autrement dit, aucun adversaire PPT $\mathcal{A}$ ne peut distinguer si la signature obtenue correspond au message m_0 ou au message m_1 , avec un avantage non négligeable.

2.6.2 Preuve : Blindness du schéma de signature

On veut montrer que, pour deux messages m_0 et m_1 , un signataire malveillant $\mathcal{A}$ ne peut pas distinguer lequel des deux messages a été signé par l'utilisateur. Autrement dit, les vues de $\mathcal{A}$ dans les deux exécutions sont indistinguables :

$$\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1).$$

Cela signifie que, pour tout adversaire PPT $\mathcal{A}$, l'avantage de distinguer le bit b choisi par l'utilisateur est négligeable :

$$\left| \Pr[b' = b] - \frac{1}{2} \right| = \text{negl}(\lambda), \text{ où } b' \text{ est le bit deviné par l'adversaire.}$$

Pour montrer cette propriété, nous construisons une suite de jeux.

G₀ :

Le premier jeu correspond à l'exécution réelle du protocole avec le message m_0 . L'utilisateur génère les clés OT $(ek_{i,0}, ek_{i,1})$ pour chaque bit du message m_0 et les envoie au signataire. Il obtient ensuite une réponse du signataire, effectue le déchiffrement et produit la signature.

↓ *On remplace les preuves NIZK par des preuves simulées produites par un simulateur de type Subversion Zero-Knowledge (sub-ZK).*

G₁ :

Ce jeu est identique à G_0 , à l'exception que les preuves NIZK sont désormais simulées. Par définition de la propriété sub-ZK, même si les paramètres sont choisis (de manière malveillante ou non) par $\mathcal{A}$, les preuves simulées sont indistinguables des preuves réelles. Ainsi, $G_0 \approx G_1$.

↓ *On applique la propriété sub-ZK à la preuve NIZK de randomisation afin de supprimer la dépendance à $F(M)$ et t' .*

G₂ :

Ce jeu est identique à G_1 , à l'exception de la preuve NIZK utilisée pour montrer la validité de la randomisation. Cette preuve est désormais simulée par le simulateur sub-ZK. La randomisation réelle utilise :

$$\hat{\sigma}_1 = \sigma_1 F(M)^{t'}, \quad \hat{\sigma}_2 = \sigma_2 g_2^{t'}.$$

Ici, le simulateur utilise une relation équivalente indépendante du message :

$$\hat{\sigma}_1 = \sigma_1 F(\mathbf{0})^{t_0}, \quad \hat{\sigma}_2 = \sigma_2 g_2^{t_0},$$

Car d'après la propriété de randomisation de Waters démontrée dans la section 2.6.3, l'aléa final $t + t'$ est uniformément distribué dans $\mathbb{Z}_p$ et peut être remplacé par un nouvel aléa $t_0 \xleftarrow{\$} \mathbb{Z}_p$ indépendant. Le remplacement de $F(M)$ par $F(\mathbf{0})$ est justifié par une réduction à l'hypothèse Decisional Diffie–Hellman (DDH), dont la preuve est présentée dans la section 2.6.4. La signature finale produite par le simulateur devient alors :

$$\sigma = (F(\mathbf{0}), \hat{\sigma}_1, \hat{\sigma}_2).$$

Comme la preuve simulée est indistinguable d'une preuve réelle, la vue du signataire reste indistinguable entre les deux jeux. Ainsi $G_1 \approx G_2$.

↓ *On simule les composantes $\sigma_{1,i}$ de manière à les rendre indépendantes du message tout en conservant leur produit égal à σ_1 .*

G₃ :

Dans le jeu précédent, la valeur σ_1 a été simulée de manière indépendante du message grâce au remplacement de $F(M)$ par $F(\mathbf{0})$. Il reste toutefois à enlever la dépendance au message dans chacune des composantes $\sigma_{1,i}$, car la signature vérifie

$$\sigma_1 = \prod_{i=1}^{\ell} \sigma_{1,i}.$$

Le simulateur choisit donc uniformément $\sigma_{1,1}, \dots, \sigma_{1,\ell-1}$, puis définit

$$\sigma_{1,\ell} = \sigma_1 \left(\prod_{i=1}^{\ell-1} \sigma_{1,i} \right)^{-1}.$$

Ainsi, $\prod_{i=1}^{\ell} \sigma_{1,i} = \sigma_1$, et la relation est préservée. Comme σ_1 est déjà indépendante du message dans G_2 , la distribution des composantes simulées $\sigma_{1,1}, \dots, \sigma_{1,\ell}$ l'est également. Les jeux G_2 et G_3 sont donc identiquement distribués, et donc $G_2 \approx G_3$.

↓ *On utilise la propriété OT hiding pour montrer que la distribution des clés OT ne révèle aucune information sur le message.*

G₄ :

En effet, si $M_i = 0$, $(ek_{i,0}, ek_{i,1}) = (g_1^{y_i}, g_1^{x_i - y_i})$, et si $M_i = 1$, $(ek_{i,0}, ek_{i,1}) = (g_1^{x_i - y_i}, g_1^{y_i})$. Comme y_i est choisi uniformément dans $\mathbb{Z}_p$, la valeur $x_i - y_i$ l'est également. Les deux distributions sont donc identiques et ne révèlent aucune information sur les bits M_i . Ainsi, $G_3 \approx G_4$.

↓ *On remplace le message m_0 par le message m_1 .*

G₅ :

Ce jeu correspond à l'exécution réelle du protocole avec le message m_1 .

Conclusion

Par les transitions effectuées entre les différents jeux, on obtient :

$$G_0 \approx G_1 \approx G_2 \approx G_3 \approx G_4 \approx G_5.$$

Ainsi, la vue du signataire malveillant dans l'exécution du protocole avec le message m_0 est indistinguable de celle obtenue dans l'exécution avec le message m_1 :

$$\text{View}_{\mathcal{A}}(m_0) \approx \text{View}_{\mathcal{A}}(m_1).$$

Par conséquent, l'adversaire $\mathcal{A}$ ne peut pas distinguer quel message a été signé par l'utilisateur. Le protocole satisfait donc la propriété de blindness.

2.6.3 Preuve : randomisation de Waters ($G_1 \rightarrow G_2$)

La signature obtenue après le déchiffrement est une signature de Waters de la forme :

$$\sigma_1 = h_s^a \cdot F(M)^t, \quad \sigma_2 = g_2^t,$$

où $t \leftarrow \mathbb{Z}_p$ est un aléa choisi lors de la génération de la signature. Lors de l'étape de randomisation, l'utilisateur choisit un nouvel aléa $t' \leftarrow \mathbb{Z}_p$ et calcule :

$$\begin{aligned} \hat{\sigma}_1 &= \sigma_1 F(M)^{t'} = h_s^a F(M)^t F(M)^{t'} = h_s^a F(M)^{t+t'}, \\ \hat{\sigma}_2 &= \sigma_2 g_2^{t'} = g_2^t g_2^{t'} = g_2^{t+t'}. \end{aligned}$$

Or t' est choisi uniformément dans $\mathbb{Z}_p$, donc $t + t'$ est également uniformément distribuée dans $\mathbb{Z}_p$. La valeur de l'aléa final $t + t'$ est indépendante de l'aléa initial t et masque la dépendance du message dans la signature. La randomisation permet donc de générer une signature dont la distribution ne révèle aucune information supplémentaire sur le message. Cette propriété montre que la randomisation d'une signature de Waters produit une distribution indépendante de l'aléa initial de signature. Elle permet donc de remplacer, dans la transition entre G_1 et G_2 , la randomisation réelle par une randomisation simulée utilisant un aléa indépendant du message.

2.6.4 Preuve : Réduction DDH ($G_1 \rightarrow G_2$)

Nous montrons que la transition entre les jeux G_1 et G_2 est computationnellement indistinguable sous l'hypothèse Decisional Diffie-Hellman (DDH). Le simulateur reçoit une instance DDH (g, g^a, g^b, c) , où il doit distinguer les deux cas suivants :

$$c = g^{ab} \quad \text{ou} \quad c \xleftarrow{\$} G.$$

À partir de cette instance, le simulateur exécute l'algorithme **Setup** et construit les paramètres publics (les bases $u_0, u_1, \dots$) tel que :

- si $c = g^{ab}$, la distribution simulée correspondre au jeu contenant $F(M)$;
- si c est uniforme dans G , la distribution simulée correspondre au jeu contenant $F(\mathbf{0})$.

Ainsi, tout adversaire capable de distinguer ces deux jeux permet de résoudre le problème DDH avec un avantage non négligeable.

3 Protocole basé sur Dual-Mode OTs

3.1 Paramètres publics

- un système de groupes bilinéaires $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, où $\mathbb{G}_1$ et $\mathbb{G}_2$ sont des groupes cycliques d'ordre premier p , munis d'un couplage bilinéaire $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, avec des générateurs $g_1 \in \mathbb{G}_1$ et $g_2 \in \mathbb{G}_2$.
- les paramètres de la fonction de hachage de Waters, constitués d'un élément aléatoire $u_0 \in \mathbb{G}_1$ ainsi que d'un vecteur $\mathbf{u} = (u_1, \dots, u_\ell) \in \mathbb{G}_1^\ell$. La fonction de hachage associée est toujours :

$$F(M) = u_0 \prod_{i=1}^{\ell} u_i^{M_i}.$$

- un CRS global pour l'OT dual-mode : $\text{OT-CRS} = (g, h, w, \tilde{w})$, où les éléments sont choisis uniformément dans le groupe approprié. On définit également : $\eta = \log_g(h)$, lorsque $g \neq 1$.
- la clé publique du schéma de signature : $\text{bvk} = g_2^a$, associée à la clé secrète : $\text{bsk} = h_s^a$, où $h_s \in \mathbb{G}_1$ est choisi aléatoirement.

3.2 Schéma de signature

Signer	User
	$\forall i \in [\ell] : y_i \xleftarrow{\$} \mathbb{Z}_p$ $(\text{ek}_{i,M_i}, \tilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i})$ $(\text{ek}_{i,1-M_i}, \tilde{\text{ek}}_{i,1-M_i}) = (w \cdot \text{ek}_{i,M_i}^{-1}, \tilde{w} \cdot \tilde{\text{ek}}_{i,M_i}^{-1})$ $(\text{ek}_{i,0}, \tilde{\text{ek}}_{i,0})_{i \in [\ell]}$
$t \xleftarrow{\$} \mathbb{Z}_p$ $\forall i \in [\ell] : (\text{ek}_{i,1}, \tilde{\text{ek}}_{i,1}) = \left(\frac{w}{\text{ek}_{i,0}}, \frac{\tilde{w}}{\tilde{\text{ek}}_{i,0}} \right)$ $\forall i \in [\ell] : \text{choisir } \alpha_i \text{ tel que } \prod_i \alpha_i = \text{bsk } u_0^t$ $\forall i \in [\ell], b \in \{0, 1\} :$ $s_{i,b}, s'_{i,b} \xleftarrow{\$} \mathbb{Z}_p$ $C_{i,b} = g^{s_{i,b}} h^{s'_{i,b}}$ $D_{i,b} = \text{ek}_{i,b}^{s_{i,b}} \tilde{\text{ek}}_{i,b}^{s'_{i,b}} \alpha_i u_i^{tb}$ $\sigma_2 = g_2^t, \quad \sigma'_2 = g_1^t$ $(C_{i,b}, D_{i,b})_{i,b}, \sigma_2, \sigma'_2$	$e(\sigma'_2, g_2) \stackrel{?}{=} e(g_1, \sigma_2)$ $\sigma_{1,i} = D_{i,M_i} C_{i,M_i}^{-y_i}$ $\sigma_1 = \prod_{i \in [\ell]} \sigma_{1,i}$ $t' \xleftarrow{\$} \mathbb{Z}_p$ $\hat{\sigma}_1 = \sigma_1 F(M)^{t'}$ $\hat{\sigma}_2 = \sigma_2 g_2^{t'}$ $\hat{\sigma}'_2 = \sigma'_2 g_1^{t'}$ return $\sigma = (\hat{\sigma}_1, \hat{\sigma}_2, \hat{\sigma}'_2)$

3.3 Correctness

$$\text{Le vérifieur accepte} \iff \begin{cases} e(\hat{\sigma}'_2, g_2) = e(g_1, \hat{\sigma}_2), \\ e(\hat{\sigma}_1, g_2) = e(h_s, \text{bvk}) \cdot e(F(M), \hat{\sigma}_2). \end{cases}$$

Première équation (en utilisant la bilinéarité) :

$$\begin{aligned}
e(\widehat{\sigma}_2', g_2) &= e(g_1^{t+t'}, g_2) \\
&= e(g_1, g_2)^{t+t'} \\
&= e(g_1, g_2^{t+t'}) \\
&= e(g_1, \widehat{\sigma}_2).
\end{aligned}$$

Seconde équation (toujours pas bilinéarité) :

$$\begin{aligned}
e(\widehat{\sigma}_1, g_2) &= e(h_s^a F(M)^{t+t'}, g_2) \\
&= e(h_s^a, g_2) e(F(M)^{t+t'}, g_2) \\
&= e(h_s, g_2^a) e(F(M), g_2^{t+t'}) \\
&= e(h_s, \text{bvk}) e(F(M), \widehat{\sigma}_2),
\end{aligned}$$

3.4 Branches DH et messy

Pour chaque position i du message, l'utilisateur construit deux branches. La première correspond à son vrai bit M_i et est définie par

$$(\text{ek}_{i,M_i}, \widetilde{\text{ek}}_{i,M_i}) = (g^{y_i}, h^{y_i}),$$

où y_i est choisi aléatoirement. Comme $h = g^\eta$, on obtient

$$h^{y_i} = (g^{y_i})^\eta,$$

c'est-à-dire

$$\widetilde{\text{ek}}_{i,M_i} = \text{ek}_{i,M_i}^\eta.$$

Cette branche vérifie donc la relation de Diffie–Hellman et est appelée *branche DH*. C'est la branche sur laquelle le protocole fonctionne normalement. La seconde branche est construite à partir de la première :

$$(\text{ek}_{i,1-M_i}, \widetilde{\text{ek}}_{i,1-M_i}) = \left(\frac{w}{\text{ek}_{i,M_i}}, \frac{\widetilde{w}}{\widetilde{\text{ek}}_{i,M_i}} \right).$$

Pour que cette branche soit également une paire DH, il faudrait que

$$\widetilde{\text{ek}}_{i,1-M_i} = \text{ek}_{i,1-M_i}^\eta.$$

Or,

$$\text{ek}_{i,1-M_i}^\eta = \left(\frac{w}{\text{ek}_{i,M_i}} \right)^\eta = \frac{w^\eta}{\widetilde{\text{ek}}_{i,M_i}}.$$

Ainsi, cette égalité serait satisfaite uniquement si

$$\widetilde{w} = w^\eta.$$

Cependant, le CRS est choisi de manière à vérifier

$$\widetilde{w} \neq w^\eta.$$

La seconde branche ne peut donc jamais être une paire DH. C'est une *branche messy*. En résumé, pour un utilisateur honnête, chaque position contient exactement une branche DH (la bonne branche, correspondant au bit du message) et une branche messy (la mauvaise branche). Elle ne transporte aucune information utile.